

KV Store

Lezione conclusiva

Architetture dei Sistemi Distribuiti

Lezione 19

Abbiamo usato un KV store perche':

- ha un'interfaccia piccola;
- permette esperimenti rapidi;
- fa emergere problemi reali dei sistemi distribuiti;
- obbliga a separare interfaccia, contratto e implementazione.

Il punto non era costruire un database completo. Il punto era costruire una palestra in cui ogni nuova feature cambia il significato delle promesse del sistema.

Ogni tappa del percorso ha riproposto la stessa domanda:

che cosa puo' legittimamente osservare il client?

Questa domanda e' piu' precisa di:

- il codice funziona?
- la risposta e' OK?
- la chiave e' nel dizionario?

Nei sistemi distribuiti il comportamento osservabile e' il contratto.

Tappa	Problema introdotto
Nodo singolo	interfaccia e semantica delle risposte
Dispatch table	estendibilita' del protocollo
Thread	interleaving e sezioni critiche
Persistenza	crash recovery e durabilita'
Replica	ack locale o replicato
Failover	autorita' e sospetto di guasto
Quorum	letture/scritture come vincoli numerici
Sharding	posizione della chiave
Rebalancing	topologia dinamica
CAS	scrittura condizionale
Capstone	integrazione e review del contratto

Nodo Singolo: Il Contratto Nasce Subito

Nella prima versione il sistema sembrava banale:

- riceve una stringa;
- aggiorna un dizionario;
- risponde al client.

Ma già' qui abbiamo dovuto decidere:

- cosa significa GET su chiave assente;
- cosa restituisce DELETE;
- se KEYS fa parte del contratto;
- quali errori sono sintattici e quali semantici.

Con piu' thread, la domanda diventa:

quali interleaving sono realmente attraversabili?

INCR ha mostrato che read-modify-write non e' atomico per definizione:

- due thread possono leggere lo stesso valore;
- entrambi possono calcolare il successivo;
- uno degli incrementi puo' sparire.

La safety non e' un'etichetta astratta: e' un insieme di stati vietati.

Persistenza: Cambia Il Significato Di OK

Prima della persistenza:

`OK = aggiornato in memoria`

Dopo la persistenza:

`OK = ricostruibile dopo crash locale`

Questa differenza obbliga a ragionare su:

- ordine tra RAM e disco;
- write-ahead log;
- `fsync`;
- recovery idempotente.

Replica: Una Scrittura, Due Interpretazioni

Con primary-secondary, OK puo' significare:

- scritto solo sul primary;
- scritto anche sul secondary;
- scritto sul primary e messo in coda per replica futura.

La replica asincrona migliora latenza e disponibilita', ma espone letture stantie e perdita dopo crash. La replica sincrona rafforza il contratto, ma rende il sistema dipendente dal secondary.

Failover: Il Guasto E' Un'Interpretazione

Un heartbeat mancante non dimostra sempre un crash. Puo' indicare:

- nodo guasto;
- rete lenta;
- scheduler in ritardo;
- partizione.

Il timeout e' quindi una scelta di contratto operativo:

- breve: failover rapido, piu' falsi positivi;
- lungo: meno falsi positivi, recupero piu' lento.

Quorum: Il Contratto Diventa Numerico

Con N repliche:

- W decide quante repliche devono accettare una scrittura;
- R decide quante repliche devono rispondere a una lettura.

La condizione:

$$R + W > N$$

serve a garantire intersezione tra letture e scritture. Non basta avere molte copie. Bisogna specificare come vengono usate.

Sharding: Dov'e' La Chiave?

Con lo sharding la domanda centrale cambia:

quale nodo deve ricevere questa operazione?

Da qui emergono nuovi aspetti del contratto:

- il router diventa parte del sistema;
- WHERE rende osservabile la destinazione teorica;
- KEYS diventa operazione globale;
- hotspot e distribuzione dei dati diventano problemi concreti.

Rebalancing: Routing E Possesso Divergono

Quando aggiungiamo uno shard:

- cambia la funzione di routing;
- i dati già scritti restano dove sono;
- alcune letture possono interrogare il nuovo target prima della migrazione.

Quindi una chiave ha due posizioni da distinguere:

- destinazione teorica;
- posizione reale.

REBALANCE e' un protocollo, non una copia banale.

CAS: Il Client Porta Una Precondizione

CAS cambia il linguaggio dell'interfaccia. Il client non dice piu':

```
scrivi questo valore
```

ma:

```
scrivi questo valore se la versione osservata e' ancora valida
```

`version_mismatch` e' un risultato del contratto: la precondizione non e' piu' vera.

Capstone: Stato Osservabile Che Migra

La capstone combina:

- sharding;
- rebalancing;
- versioni;
- CAS;
- test di accettazione.

La parte interessante non e' solo migrare il valore. La parte interessante e' migrare lo stato osservabile:

`(value, version)`

Per ogni tappa possiamo chiederci:

che cosa non deve mai accadere?

Esempi:

- perdere una scrittura ackata;
- accettare due CAS incompatibili;
- avere due primary attivi;
- dichiarare conclusa una migrazione incompleta;
- restituire una versione non coerente con il valore.

La domanda complementare e':

che cosa deve prima o poi riuscire ad accadere?

Esempi:

- un client deve ricevere risposta;
- il recovery deve terminare;
- una replica lenta non deve bloccare sempre tutto;
- una migrazione non deve restare aperta indefinitamente;
- i retry di CAS non devono portare a starvation sistematica.

La Triade Da Non Confondere

- **Interfaccia:** cio' che il client puo' invocare.
- **Contratto:** cio' che il sistema promette osservabilmente.
- **Implementazione:** il meccanismo scelto per mantenere la promessa.

La stessa risposta testuale puo' avere significati diversi. NOT_FOUND puo' voler dire:

- chiave mai esistita;
- replica non aggiornata;
- routing nuovo, dato non ancora migrato;
- contratto di lettura debole.

Il percorso resta didattico. Un sistema reale richiederebbe ancora:

- consenso o leader election formale;
- log replicato;
- riconfigurazione sicura;
- snapshot e compaction;
- gestione robusta della membership;
- metriche, backpressure e fault injection;
- sicurezza e controllo degli accessi.

Domande Per Valutare Una Soluzione

Di fronte a una nuova proposta, chiedere sempre:

- ❶ Che cosa vede il client?
- ❷ Quale promessa viene fatta?
- ❸ Quale stato interno serve per mantenerla?
- ❹ Quali crash o interleaving possono romperla?
- ❺ Cosa succede se una parte del sistema e' lenta?
- ❻ Quale test dimostra la promessa?

Se manca una risposta, manca una parte del progetto.

Nel discutere la capstone, guardiamo soprattutto:

- chiarezza del contratto;
- conservazione di valore e versione;
- comportamento tra `ADD_SHARD` e `REBALANCE`;
- CAS dopo migrazione;
- test che colpiscono casi critici, non solo casi felici.

Il codice va giudicato insieme alla promessa che dichiara di mantenere.

Il KV store non era il fine. Era un modo compatto per far emergere le domande fondamentali:

- chi sa cosa?
- quando lo sa?
- chi e' autorizzato a rispondere?
- quale storia degli eventi diventa osservabile?
- quali promesse valgono anche con crash, concorrenza e topologia dinamica?

Queste domande restano le stesse anche in sistemi molto piu' grandi.